

1 Introduction

As discussed in the coursework introduction lecture, encryption and cybersecurity has been the most undervalued technology that was likely to be adopted by 2025. This clearly showed the demand for this technology is important moving forward. Furthermore, there is also an increase in research and demand for devices that are smaller, faster and have a lower power utilisation.

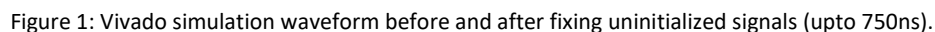
2 Project Aim and Objectives

This report aims to combine the two technologies to create a high-performance low power cryptography system that can be deployed onto an FPGA. It evaluates the performance of the implementation through simulation and physically onto the board. The objectives of this report are to design, implement, and validate a hardware-based cryptographic system on an FPGA platform using Vivado 2019.1. This begins with part 1 the development and implementation of the eXtended Tiny Encryption Algorithm (XTEA) in VHDL, supporting 128-bit block and key sizes, this is verified against the provided C reference algorithm using the supplied testbench. This is then used within a cryptography system which consists of IPs coded in system Verilog that generates encryption and decryption data. The mini router which handles the selection of which link should be granted access to the XTEA cryptography block. The output of the mini router is fed into the XTEA block which encrypts or decrypts the data as required. All the stages in part 2 aim to replicate a real world system networking system.

3 Contents

1	INTRODUCTION	1
2	PROJECT AIM AND OBJECTIVES	1
3	CONTENTS	1
4	PART 1: XTEA DEVELOPMENT AND SIMULATION	2
4.1	INFERENCE TIME	2
4.2	OUTPUT WAVEFORM FROM SIMULATION	2
5	PART 1: XTEA HARDWARE DEPLOYMENT	3
5.1	INFERENCE TIME	4
5.2	AREA AND POWER UTILISATION	4
6	PART 2: CRYPTOGRAPHY SYSTEM DEVELOPMENT AND SIMULATION	5
6.1	MINI ROUTER.....	5
6.1.1	<i>Inference Time</i>	5
6.2	IP ENC AND DEC GENERATOR WITH MINI ROUTER.....	6
6.2.1	<i>Inference Time</i>	6
7	REFERENCES	7

A five state (IDLE, LOAD, COMPUTE, OUTPUT_DATA, OUTPUT_DONE) FSM was used to implement the simplex XTEA algorithm. This is because when compared to duplex the simplex has a reduced footprint on an FPGA board, this ensures that it can be deployed on a wide array of resource constrained devices.

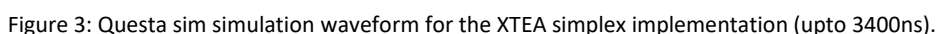


4.1 Inference Time

$$\text{Inference Time} = \text{Number of Cycles} \times \text{Clock Period} \quad (1)$$

Figure 2: Snapshot of Tcl console showing the inference time for all tests to be complete.

4.2 Output Waveform from Simulation



25WSC354 - Electronic System Design with FPGAs F330590

Figure 3 indicated that the desired output for the simplex implementation was achieved. For robustness the algorithm can be loaded onto vivado manually or running the vsim -do command on a linux terminal to open the file in questa sim. Figure 1 and 3 indicated that the algorithm worked as intended on both softwares. This was validated by using the Tcl console which gave a pass/fail for each key pair and if they matched.

5 Part 1: XTEA Hardware Deployment

During the synthesis and the implemented design for the deployment of the XTEA algorithm onto the FPGA board, there was an issue with the slack time. The negative slack found in Figure 3 suggested that the algorithm could not finish a portion of the calculation within the defined clock period. Negative slack time can cause stale data being used for the next computation.

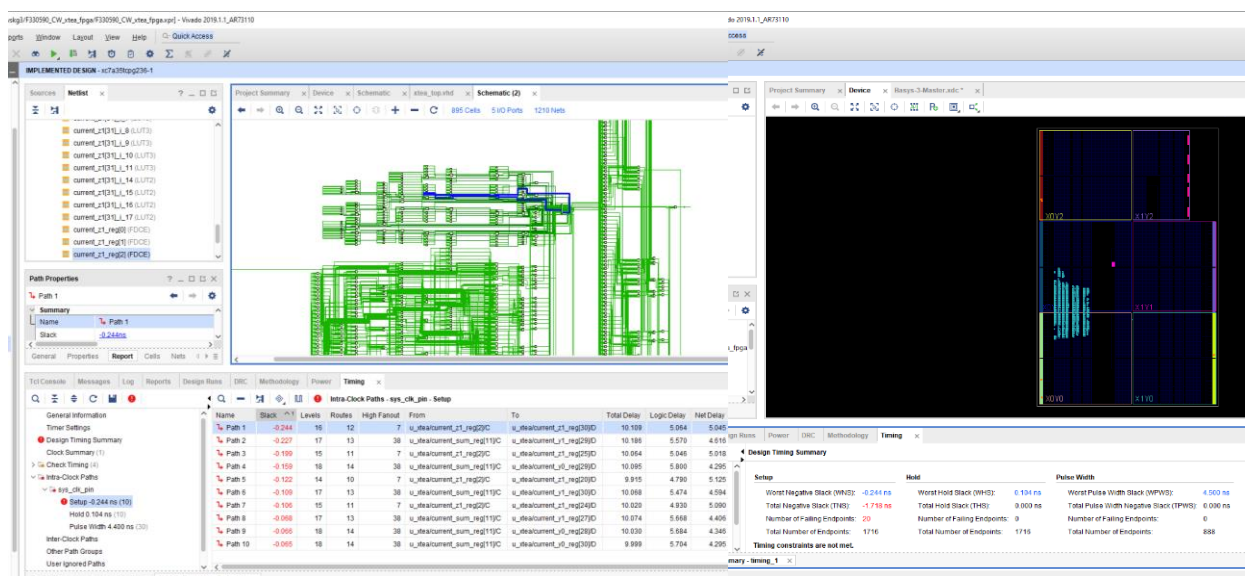


Figure 5: Implemented design showing the negative slack time.

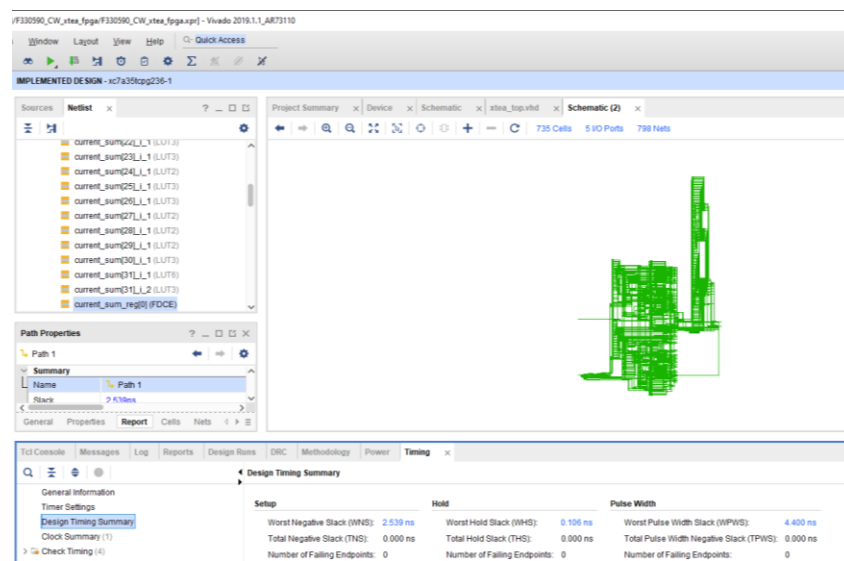
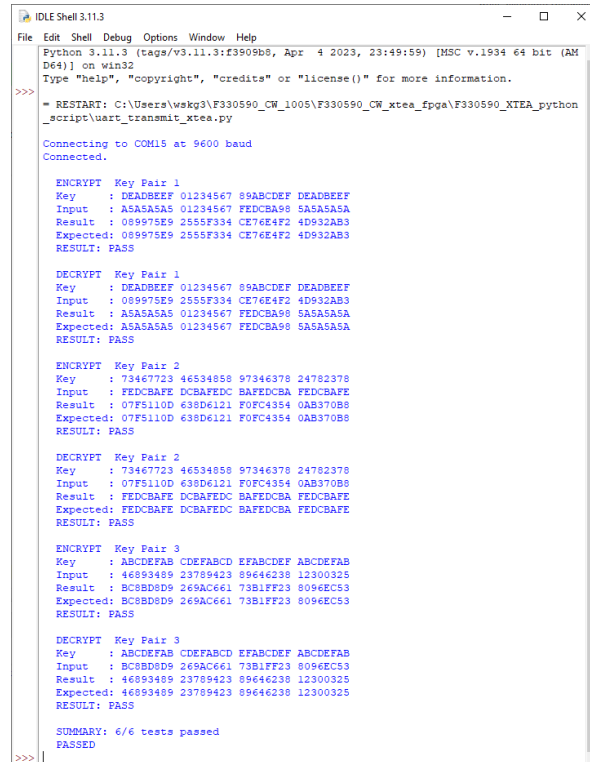


Figure 4: Implemented design showing negative slack time was fixed.

There are two proposed methods to fixing this. The simplest fix was to increase the clock period. However, this would slow down the algorithm, as it would decrease the clock frequency. Thus, pipelining the

25WSC354 - Electronic System Design with FPGAs F330590

compute stage was considered as the better choice, this does not have any impact on the clock period. New signals were added and the clock cycles taken to achieve the compute stage was doubled. This is a trade-off to meeting the desired slack time. Figure 5 shows the improvement that this made, this is because the number of operations in the critical path were successfully reduced.



```
IDLE Shell 3.11.3
Python 3.11.3 (tags/v3.11.3:f3909b8, Apr 4 2023, 23:49:59) [MSC v.1934 64 bit (AMD64)] on win32
Type "help()", "copyright()", "credits()" or "license()" for more information.
>>>
= RESTART: C:\Users\wskg3\F330590_CW_1005\F330590_CW_xtea_fpga\F330590_XTEA_python_script\uart_transmit_xtea.py
Connecting to COM15 at 9600 baud
Connected.

ENCRYPT Key Pair 1
Key : DEADBEEF 01234567 89ABCDEF DEADBEEF
Input : ASASASAS 01234567 FEDCBA98 5A5A5A5A
Result : 089975E9 2555F334 CE76E4F2 4D932AB3
Expected: 089975E9 2555F334 CE76E4F2 4D932AB3
RESULT: PASS

DECRYPT Key Pair 1
Key : DEADBEEF 01234567 89ABCDEF DEADBEEF
Input : 089975E9 2555F334 CE76E4F2 4D932AB3
Result : ASASASAS 01234567 FEDCBA98 5A5A5A5A
Expected: ASASASAS 01234567 FEDCBA98 5A5A5A5A
RESULT: PASS

ENCRYPT Key Pair 2
Key : 73467723 46534858 97346378 24782378
Input : FEDCBAFE DCBAFEDC BAFEDCBA FEDCBAFE
Result : 07F5110D 638D6121 F0FC4354 0AB370B8
Expected: 07F5110D 638D6121 F0FC4354 0AB370B8
RESULT: PASS

DECRYPT Key Pair 2
Key : 73467723 46534858 97346378 24782378
Input : 07F5110D 638D6121 F0FC4354 0AB370B8
Result : FEDCBAFE DCBAFEDC BAFEDCBA FEDCBAFE
Expected: FEDCBAFE DCBAFEDC BAFEDCBA FEDCBAFE
RESULT: PASS

ENCRYPT Key Pair 3
Key : ABCDEFAB CDEFABCD EFABCD EFABCD
Input : 46893489 23789423 89646238 12300325
Result : BC8BD8D9 268AC661 73B1FF23 8096EC53
Expected: BC8BD8D9 268AC661 73B1FF23 8096EC53
RESULT: PASS

DECRYPT Key Pair 3
Key : ABCDEFAB CDEFABCD EFABCD EFABCD
Input : BC8BD8D9 268AC661 73B1FF23 8096EC53
Result : 46893489 23789423 89646238 12300325
Expected: 46893489 23789423 89646238 12300325
RESULT: PASS

SUMMARY: 6/6 tests passed
PASSED
>>>
```

Figure 6: IDLE output window from python script showing the encryption and decryption match using uart.

Once the negative slack time was solved a python script was developed so that the input data could be transmitted. This was done as the simulation model took up 95% of the IO, this was because the values for testing in simulation were hard coded and taking up unnecessary resources. Therefore, the uart functionality was implemented, with the assistance from [1]. An additional VHDL file was created with an eight state FSM which handled the loading of encrypted/decrypted data and then extracting the output from the XTEA block. Figure 6 indicated that the XTEA crypto block worked as intended. LED(15) to LED(10) is also used as a hardware check, the LEDs turn on as each test passes, they remain on at the end of the program. Note that there is a purposeful one second delay between each test. This provides both hardware and software validation for the hardware deployment of the XTEA algorithm.

5.1 Inference Time

Using the time module in python, you are able to obtain the inference time for the XTEA algorithm once it has been deployed. The average time that it took was 60,266,317 ns, which is about 0.0603 seconds, this includes the time it takes for the data to be transmitted via the rx and tx ports.

5.2 Area and Power Utilisation

Figure 7 and 8 indicated that there was a decrease in on chip power consumption once pipelining was introduced, this is also indicated by the increase in flip flops used when comparing the two figures. It also

suggested that there would be a higher frequency for the pipelined implementation compared to the non-pipelined.

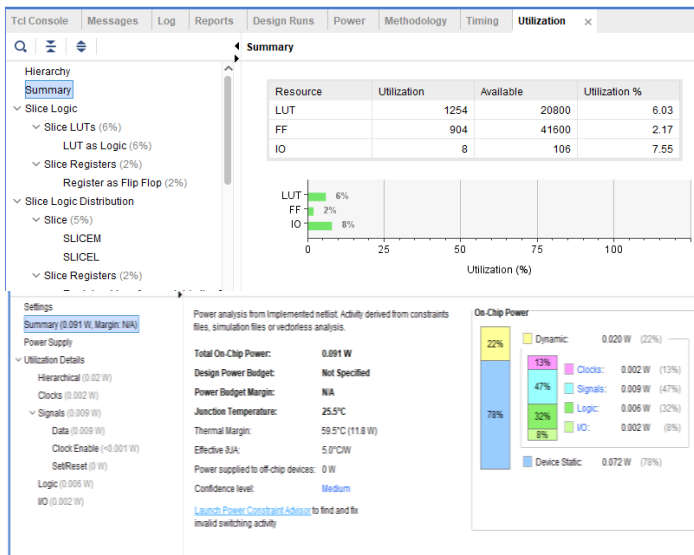


Figure 7: Screenshots of area and power metrics for part 1 XTEA deployment before pipelined compute stage.

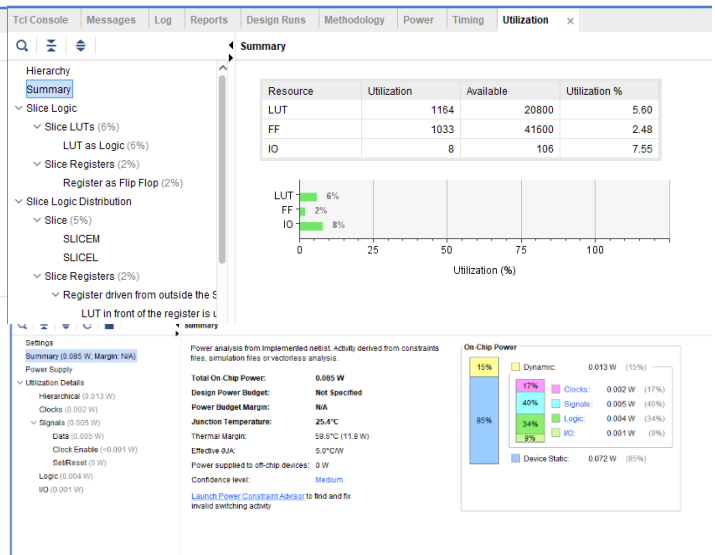


Figure 8: Screenshots of area and power metrics for part 1 XTEA deployment after pipelined compute stage.

6 Part 2: Cryptography System Development and Simulation

6.1 Mini Router

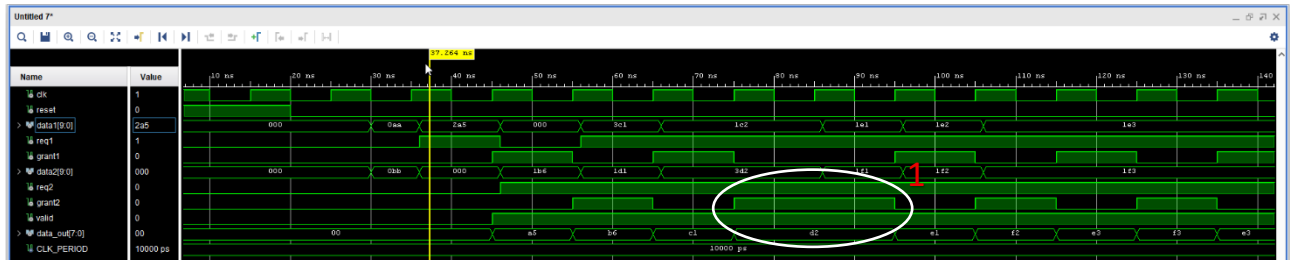


Figure 9: Vivado simulation waveform for mini router implementation (upto 140ns).

The testbench consisted of eight tests. Figure 9 indicated that the mini router implementation successfully selected the right link for each test. For example, at 30 ns, since no request is high neither data is granted access to be outputted. However, at 40 ns req1 goes high this is why at 45 ns the data output is valid and grant 2 goes high.

However, there was a slight increase in the time 0xd2 (denoted by 1 in Figure 9) is output for between 75 ns and 95 ns, this delay might be due to the extra processing time that is needed to decide which link should be chosen using the round robin approach. Therefore, the longer grant and data_out signal is reasonable.

There was an issue with uninitialised signals, this was fixed with the implementation of an asynchronous reset, similar to how this problem was fixed for the XTEA part.

6.1.1 Inference Time

The mini router on its own had a inference time of 146 ns.

25WSC354 - Electronic System Design with FPGAs F330590

Using equation 1, the mini router takes 10 clock cycles to complete all test cases.

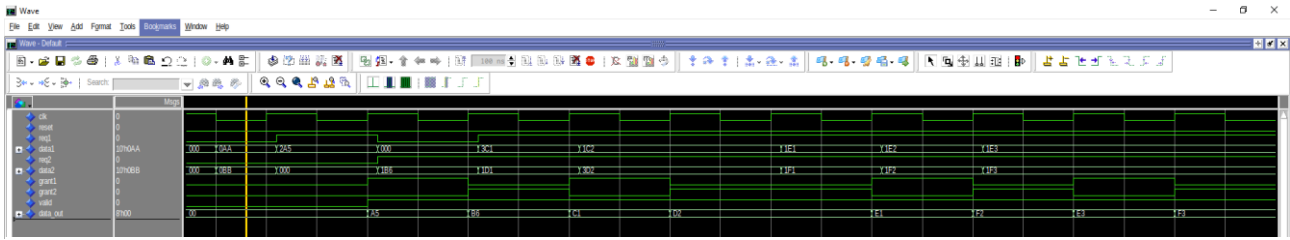


Figure 10: Questa sim simulation waveform for the mini router (upto 130 ns).

6.2 IP Enc and Dec Generator with Mini Router

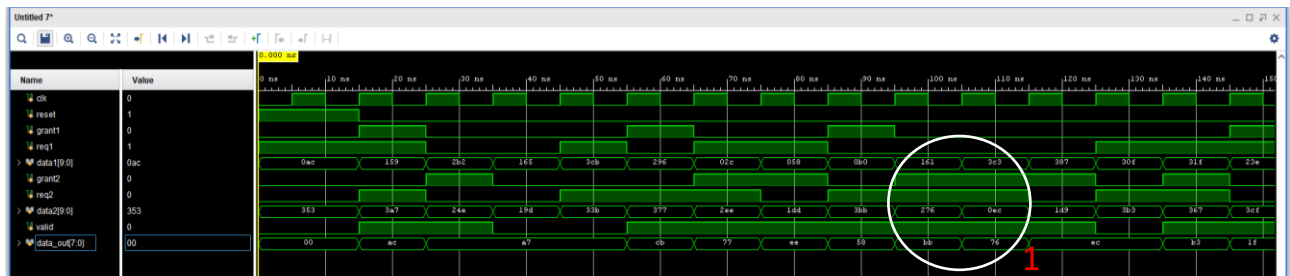


Figure 11: Vivado simulation waveform for the enc and dec data being selected using the mini_router (for upto 150ns).

The two IPs are responsible for generating data using the XTEA algorithm in part 1 so it can simulate real world encryption and decryption. The enc generator produces link 1, which consists of 12 bits (8 for data, 2 for priority bits, 1 for request signal and another to store grant signal that is received from the mini router). The dec generator produces the same output. They both use an 8 bit linear feedback shift register (LFSR), to simulate random number generation, this is because they are considered cheap and therefore are resource efficient.

```
req1 = 0 data1_pri = 1 data1_byte = 0x61 grant1 = 0 | req2 = 1 data2_pri = 2 data2_byte = 0x76 grant2 = 1 | valid = 1 data_out = 0xbb  
req1 = 0 data1_pri = 3 data1_byte = 0xc3 grant1 = 0 | req2 = 1 data2_pri = 0 data2_byte = 0xec grant2 = 1 | valid = 1 data_out = 0x76
```

Figure 12: Snapshot of the vivado Tcl console for annotated section 1.

```
req1 = 1 data1_pri = 3 data1_byte = 0x6f grant1 = 0 | req2 = 1 data2_pri = 3 data2_byte = 0xbf grant2 = 0 | valid = 0 data_out = 0xef  
req1 = 1 data1_pri = 2 data1_byte = 0xde grant1 = 1 | req2 = 1 data2_pri = 2 data2_byte = 0x7e grant2 = 0 | valid = 1 data_out = 0x6f  
req1 = 1 data1_pri = 1 data1_byte = 0xbd grant1 = 0 | req2 = 1 data2_pri = 1 data2_byte = 0xfd grant2 = 1 | valid = 1 data_out = 0x7e
```

Figure 13: Snapshot of the vivado tcl console showing the round robin approach working as intended.

Looking at the annotated section (1) the output of the data happens one cycle later as expected, which is 0x76, this is because the when two values 0x61 (link 1) and 0x76 (link 2) are passed into the router req 2 is the only one that is high. This is also shown in Figure 11.

Figure 12 shows that when link 1 won when both priority and req were the same for both links, link 2 won the following time. Figure 10, 11 and 12 indicated that the IPs that generated the enc and dec data and router successfully worked together as intended.

6.2.1 Inference Time

The inference time with the generated encrypted and decrypted data being fed into the mini router was 515 ns.

7 References

- [1] Hackster.io (2020). *UART Interface in VHDL for Basys3 Board*. [online] Hackster.io. Available at:
<https://www.hackster.io/alexey-sudbin/uart-interface-in-vhdl-for-basys3-board-eef170>.